1 Лекция 07. Iterative compression: от большого решения к малому

1.1 Что даёт почти подходящее решение

Итеративное сжатие (iterative compression) — метод, в котором мы постепенно строим вход и на каждом шаге исправляем уже известное решение, ставшее слишком большим. Например, для Vertex Cover известны $k+1$ вершин, покрывающих все рёбра, а нужно не больше k. Это дополнительная информация: после удаления известного покрытия не остаётся ни одного ребра.

Важно, что требуется найти **любое** меньшее покрытие. На звезде с центром v и листьями a, b, c множество $Z = \{a, b, c\}$ является покрытием. При $k = 2$ ни одно двухэлементное подмножество Z не покрывает все три ребра, но решение $\{v\}$ существует. Слово «сжатие» означает уменьшение размера решения с возможной заменой его вершин.

Формально Vertex Cover получает конечный простой граф G и бюджет k; решение пересекает каждое ребро. Для $X \subseteq V$ запись $G[X]$ означает индуцированный подграф: оставляем вершины X и все исходные рёбра между ними. $G-X$ удаляет X. В этой главе строим полный алгоритм: внешний цикл даёт почти подходящее решение, внутренний перебор уточняет его пересечение с искомым, а остаточная задача решается точно.

Это отличается от ядеризации: мы не обязаны уменьшать вход до функции параметра и не требуем полиномиального времени всего сжатия. Допустима FPT-зависимость от k.

1.2 Что именно сжимается

В iterative compression сжимают **решение**, а не входной экземпляр. Подзадача compression получает граф G, бюджет k и уже известное допустимое множество Z размера не больше $k+1$. Нужно найти любое решение размера не больше k или доказать его отсутствие. Новое решение не обязано быть подмножеством Z.

Упорядочим вершины $v_1, \ldots, v_n$ и рассмотрим $G_i = G[\{v_1, \ldots, v_i\}]$. Для Vertex Cover начинаем с $S_0 = \varnothing$. Имея покрытие S_{i-1} в G_{i-1} размера не больше k, получаем $Z = S_{i-1} \cup \{v_i\}$ — покрытие G_i размера не больше $k+1$. Все новые рёбра инцидентны v_i, поэтому это действительно покрытие.

Если $|Z| \le k$, идём дальше. Иначе вызываем compression. Если компрессия доказала невозможность, исходный граф тоже NO: ограничение любого его покрытия на первые i вершин покрывает G_i и не увеличивает размер. Если каждый шаг успешен, S_n — ответ.

При времени compression $f(k)N^c$ общая оценка не больше $nf(k)N^c$. Нужно учитывать и внешний цикл, хотя в O^* этот множитель скрывается.

1.3 Угадывание пересечения

Пусть существует искомое решение S. Оно пересекает Z некоторым $I \subseteq Z$. Переберём все I с $|I| \le k$. Зафиксируем их включение в решение, удалим из графа и уменьшим бюджет до $b = k - |I|$. Вершины $W = Z \setminus I$ теперь **запрещено** выбирать: мы угадали пересечение в точности.

Получилась disjoint-задача. Она часто проще, поскольку дополнение к малому известному решению уже имеет удобную структуру. Перебор охватывает настоящее $I = S \cap Z$, поэтому достаточно точно решить каждый disjoint-экземпляр. Утверждение «малое Z дано» без такого решателя ещё не FPT-алгоритм.

1.4 Disjoint Vertex Cover

Поскольку Z было покрытием, $V \setminus Z$ независимо. После удаления I единственные возможные рёбра лежат внутри W или между W и $V \setminus Z$.

Если в $G[W]$ есть ребро, ветвь невозможна: оба его конца запрещены. Если W независимо, все внешние соседи

$$F = N_G(W) \setminus Z$$

обязательны. Они и образуют достаточное покрытие остатка: других рёбер нет. Поэтому ветвь принимается в точности при $|I| + |F| \le k$, и возвращает $I \cup F$.

Корректность. В успешной ветви рёбра с концом в I покрыты I, рёбра от W наружу — F, рёбер внутри W и вне Z нет. Обратно, всякое покрытие с пересечением I обязано взять все вершины F, поэтому при превышении бюджета такая ветвь действительно невозможна.

Переборов максимум 2^{k+1}. Проверка ветви за $O(n+m)$ даёт общий алгоритм $O(2^k n(n+m))$. Для Vertex Cover это хуже хорошего ветвления, но важно как полностью разобранная схема для более трудных задач удаления.

1.5 Ручной проход внутреннего сжатия

Вернёмся к звезде с центром v, листьями a, b, c, известным $Z = \{a, b, c\}$ и бюджетом $k = 2$. Ветка $I = \varnothing$ запрещает брать все три листа. Множество $W = Z$ независимо; внешние соседи $F = \{v\}$ обязательны и достаточны. Получаем решение $\{v\}$ стоимости 1.

Ветка $I = \{a\}$ тратит единицу на a, запрещает b, c и снова заставляет взять v. Решение $\{a, v\}$ допустимо, хотя не минимально. Задача сжатия требует только размер не больше k, поэтому оно тоже подошло бы. Ветка $I = \{a, b\}$ уже тратит весь бюджет, но оставшийся запрещённый лист c заставляет добавить v; такую ветку отвергаем.

Перебор I нужен не для выбора подмножества старого ответа как окончательного решения, а для фиксации его точного пересечения с новым. Все прочие вершины нового решения ищет disjoint-подзадача. В OCT эта подзадача станет сложнее, но внешний цикл и аргумент полноты останутся теми же.

1.6 Когда схема переносится

Для удаления вершин до наследственного свойства $\mathcal{P}$ — свойства, сохраняющегося при взятии индуцированного подграфа, — внешний цикл работает так же: $(G_i - (S_{i-1} \cup \{v_i\}))$ уже обладает $\mathcal{P}$. Отрицательный ответ на префиксе исключает ответ на целом графе. Но внутренняя disjoint-задача зависит от $\mathcal{P}$ и может быть сложной.

OCT удаляет вершины до двудольности. Нельзя просто ветвиться по вершинам произвольного нечётного цикла: его длина не ограничена k. Compression даёт вместо этого малое Z, после удаления которого весь граф двудолен. Следующая лекция использует эту структуру вместе с min-cut.

Самопроверка: почему нельзя только перебирать k-подмножества Z? В каком месте применяется наследственность? Почему алгоритм compression должен возвращать само решение, а не только булев ответ, либо обеспечивать его восстановление?

1.7 Практика

Условия задач 07.1–07.5. Решения открывайте после своей попытки.

1.8 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §4.1 и §4.1.1, с. 78–80. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.